

System Design Crash Course

Complete Detailed Notes with Examples

Build a production-ready, scalable architecture from scratch: break it, fix it, repeat.

#	Chapter	Problem it solves
1	Client-Server, DNS, HTTPS	The starting point
2	Vertical vs Horizontal Scaling	One server cannot handle the load
3	Load Balancer	Client must pick among many servers
4	CDN	High latency for far-away users
5	Caching (Redis)	Database is the bottleneck
6	Database Replication	Too many reads for one DB
7	Sharding and Consistent Hashing	Too much data for one DB
8	Message Queues	Slow synchronous work blocks the user
9	Blob / Object Storage, Pre-signed URLs	Files do not belong in a database
10	Real-time: Polling, WebSockets, SSE	Live updates
11	Idempotency	Retries cause duplicate effects
12	CAP Theorem	Consistency vs availability
13	Final Architecture and Interview Cheat Sheet	Putting it all together

0. The Big Idea: How to Think About System Design

This course does not just list topics. It builds a system **from zero** and, at every step, **breaks** it (a bottleneck or failure appears), then **adds one component** to fix it. Each new component exists because it solves one specific problem.

Problem appears -> Add a component -> New problem appears -> Add another ...

1 server too small	-> scale horizontally
clients can't choose	-> load balancer
far users are slow	-> CDN
DB is overloaded	-> cache (Redis)
too many DB reads	-> read replicas
too much data	-> sharding
slow work blocks users	-> message queue + workers
files bloat the DB	-> blob storage (S3)

KEY TAKEAWAY

In an interview, never just name components. Explain **why** each one is there: 'We added X because Y was breaking.'
That is the mental model interviewers want.

1. Client-Server Architecture, DNS and HTTPS

1.1 The most basic architecture

There is a **client** (browser or mobile app) and a **server** (a remote machine running your code). The client wants to reach the server, but a human only knows a **URL** such as `example.com`, while machines talk using **IP addresses**.

1.2 Step-by-step: what happens when you open `example.com`

- **DNS resolution:** the client asks a DNS resolver, 'What is the IP of `example.com`?' The resolver answers with an IP, e.g. `192.15.16.28`. DNS is the internet's phone book (name -> number).
- **TCP connection:** the client opens a TCP connection to that IP (a reliable, ordered channel).
- **HTTP(S) request:** over this connection the client sends a request, e.g. `GET /home`.
- **Response:** the server replies with HTML/JSON and the connection serves the page.

```
Client --(1) example.com?--> DNS Resolver
Client <--(2) 192.15.16.28 -- DNS Resolver
Client --(3) TCP connect + HTTPS request--> Server (192.15.16.28)
Client <--(4) response ----- Server
```

EXAMPLE

You type **amazon.com**. Your laptop asks DNS for the IP, receives something like `205.251.242.103`, opens a TCP connection to it, sends `'GET /'` and the Amazon server returns the homepage.

1.3 HTTP vs HTTPS

Plain HTTP is **not secure**: anyone on the network (e.g. cafe Wi-Fi) can read or modify the data. **HTTPS** = HTTP + TLS encryption. In the real world **only HTTPS is used**.

EXAMPLE

Logging into a bank over HTTP would expose your password in plain text to anyone sniffing the Wi-Fi. Over HTTPS the same bytes are encrypted and unreadable.

KEY TAKEAWAY

Flow to remember: URL -> DNS -> IP -> TCP connection -> HTTPS request -> response.

2. Scaling: Vertical vs Horizontal

Initially 2-3 users per day visit the app and a single server is fine. Then the app becomes popular: 10,000 users per day, then 1 million.

2.1 Vertical scaling (scale up)

Make the **one server bigger**: add more CPU, RAM and storage.

EXAMPLE

Your 8 GB RAM server struggles at 10K users. You upgrade to 64 GB RAM and a faster CPU, and it copes. Simple, no code changes.

PROBLEM / WATCH OUT

Hard limit: a machine can only get so big. You cannot add infinite RAM. At 1 million users even the biggest machine fails.

Single point of failure (SPOF): if this one server dies, the entire service is down.

2.2 Horizontal scaling (scale out)

Instead of one huge server, add **many normal servers** running the same application.

EXAMPLE

App Server 1, 2 and 3 all run your code. If Server 2 crashes, Servers 1 and 3 still serve users, so the service stays alive.

Aspect	Vertical scaling	Horizontal scaling
Method	Bigger machine	More machines
Limit	Hard hardware ceiling	Practically unlimited
Fault tolerance	SPOF	Survives individual failures
Complexity	Low	Higher (needs load balancer, etc.)
Example	8 GB -> 128 GB RAM	1 server -> 10 servers

PROBLEM / WATCH OUT

New problem: the client knows only **one IP address**. With 3 servers, which one does it talk to? Answer: a load balancer.

3. Load Balancer

A **load balancer (LB)** sits in front of the app servers. Clients no longer talk to servers directly; they talk to the LB, which decides which server handles each request.

```
Client --> Load Balancer
                        |--> App Server 1
                        |--> App Server 2
                        |--> App Server 3
```

3.1 Load balancing algorithms

(a) Round Robin

Requests are handed to servers in a fixed rotation: R1->S1, R2->S2, R3->S3, R4->S1, R5->S2 ...

EXAMPLE

Six requests, three servers: S1 gets R1 and R4, S2 gets R2 and R5, S3 gets R3 and R6.

Best when: requests are uniform (each takes about the same time). **Weakness:** in real life requests differ in cost, so one server can get all the slow ones.

(b) Least Connections

Send the new request to the server with the **fewest active connections right now**.

EXAMPLE

R1 takes 5 s on S1, R2 takes 1 s on S2, R3 takes 3 s on S3. When R4 arrives after about 1 second, S2 has already finished, so R4 goes to S2 (the least busy).

Best when: requests are non-uniform (some heavy, some light).

(c) IP Hashing

Hash the client's IP to pick a server, so the **same client always lands on the same server**.

EXAMPLE

Client 1 -> Server 1, Client 2 -> Server 1, Client 3 -> Server 2, every time. This gives **sticky sessions**: if Server 1 stores the context of Client 1's shopping cart in memory, the next request must reach Server 1 or the cart appears empty.

Algorithm	Idea	Use when
Round Robin	Rotate through servers	Uniform requests
Least Connections	Pick least busy server	Uneven request durations
IP Hashing	Same client -> same server	Sticky sessions needed

3.2 Layer 4 vs Layer 7 load balancers

This refers to the OSI model layer at which the LB inspects traffic.

Layer 4 (Transport layer)

- Routes using only **IP address and TCP port**.
- Does not look inside the request, so it cannot tell an image request from an API call.
- **Fast and lightweight** but not smart.

Layer 7 (Application layer)

- Understands the request: **URL path, headers, cookies**.
- Can route intelligently (content-based routing).

- Modern LBs (AWS ALB, Nginx) are Layer 7.

EXAMPLE

Requests to `/api/orders` go to the application servers, while requests to `/static/logo.png` go to the file server or CDN. A cookie `beta=true` can send a user to servers running the new version. An L4 LB cannot do any of this because it only sees IP:port.

3.3 Health checks

The LB constantly pings every server: 'Are you alive?' (often an HTTP call to `/health`). If a server stops responding, the LB **removes it from rotation** and sends traffic only to healthy servers.

EXAMPLE

Health check on App Server 3 fails. New requests flow only to Servers 1 and 2. Users never notice. When Server 3 recovers and passes checks, it rejoins the rotation. This gives **high availability**.

3.4 Is the load balancer itself a single point of failure?

Yes, it could be. The fix is to run **multiple load balancers** in **active-active** (all serve traffic) or **active-passive** (a standby takes over on failure) pairs. In practice **your cloud provider handles this** (e.g. AWS ELB is managed and redundant), so you rarely worry about it.

KEY TAKEAWAY

LB = horizontal scaling enabler + health checks + high availability. Use L7 for modern apps.

PROBLEM / WATCH OUT

New problem: a user in Singapore is still served by a server in Virginia (USA). Every request travels halfway around the world = high latency. Answer: CDN.

4. CDN (Content Delivery Network)

A CDN is a **globally distributed network of servers** called **edge servers / edge nodes / Points of Presence (PoPs)**. Static content is served from the edge node nearest to the user instead of the faraway **origin server**.

EXAMPLE

Origin server is in US-East. Without CDN, a Mumbai user waits maybe 250 ms per request. With a CDN edge node in Mumbai the same image arrives in about 20 ms.

```
User (Singapore) --> Edge node (Asia)  --\
User (Mumbai)    --> Edge node (Asia)  ---+--> Origin server (US-East)
User (London)    --> Edge node (Europe) --/      (only on cache miss)
```

4.1 How a CDN works: cache hit vs cache miss

- **Cache miss:** a user in Singapore requests `logo.png`; the Asia edge node does not have it yet. It fetches the file from the origin, stores a local copy, and returns it.
- **Cache hit:** the next user in Singapore/Mumbai requests the same file; the Asia edge node serves it immediately without contacting the origin.

4.2 What belongs on a CDN?

Store on CDN (static)	Do NOT store on CDN (dynamic)
Images, videos, audio	API responses
JavaScript, CSS, fonts	User-specific data (your profile, cart)
HTML of static sites	Anything generated per request

4.3 Cache invalidation

Problem: a file changes at the origin (e.g. a new CSS), but the CDN keeps serving the old copy. Two approaches:

- **TTL (time to live):** each cached file expires automatically after N minutes/days and is re-fetched. Example: Cache-Control: max-age=86400 (1 day).
- **Versioned (fingerprinted) file names** (what production uses): `main.a1b2c3.js` -> `main.a2b2c3.js` -> `main.a2b2c4.js`. A new deployment creates a new URL, so the CDN naturally treats it as a brand-new file. Old users get the old URL, new users get the new one, and no manual purge is needed.

KEY TAKEAWAY

CDN = low latency for static content via nearby edge nodes. Invalidate using TTL or versioned filenames.

PROBLEM / WATCH OUT

New problem: dynamic requests still hit the database every time, and the DB becomes the bottleneck. Answer: caching.

5. Caching with Redis

A database query typically takes **5-50 ms**. If the same data (a user profile, product page, leaderboard) is requested thousands of times per second and rarely changes, hammering the DB is wasteful. A **cache** stores frequently accessed data **in memory (RAM)**, where access takes **nanoseconds to under a millisecond**.

Redis is the industry-standard in-memory data store. It supports rich data structures: strings, hashes, sorted sets, lists (a good thing to mention in interviews).

```
App Server --(1) check cache--> Redis
| <--(2a) HIT: return data
|
|--(2b) MISS --> Database --> (3) store in Redis --> return to client
```

EXAMPLE

Product page for iPhone 16 is read 5,000 times per second. First request: cache miss, query DB (30 ms), store result in Redis with TTL 5 min. Next 4,999 requests that second: served from Redis in about 1 ms. The DB sees 1 query instead of 5,000.

5.1 Caching (write) strategies

Write-through

Every write goes to the **cache and the database at the same time**.

- **Pro:** cache is always in sync; no data-loss risk.

- **Con:** slower writes; you pay write cost even for data that may never be read again.

EXAMPLE

User updates their name. App writes it to both Redis and DB. Any later read from cache is guaranteed fresh.

Write-back (write-behind)

Write to the **cache first**; the cache **asynchronously flushes** data to the DB in the background.

- **Pro:** extremely fast writes.
- **Con: risk of data loss:** if the cache crashes before flushing, those writes are gone forever.

EXAMPLE

A game increments a player's score 1000 times per minute in Redis and flushes to the DB every 10 seconds. Great speed, and losing 10 s of score on a crash is acceptable. For bank transfers it would not be.

Strategy	Write speed	Consistency	Data-loss risk	Typical use
Write-through	Slower	Strong	Low	Profiles, orders
Write-back	Very fast	Eventual	Higher	Counters, analytics, scores

There is always a **trade-off**: choose based on how much staleness / loss your use case tolerates.

5.2 Eviction policies (what to remove when the cache is full)

- **LRU (Least Recently Used):** evict the item not accessed for the longest time. *Default choice.* Example: cache holds A, B, C. A was last used 10 min ago, B 1 min ago, C 2 min ago. A new key arrives, so A is evicted.
- **LFU (Least Frequently Used):** evict the item accessed the fewest times overall. Example: A used 100 times, B 3 times, C 50 times. B is evicted even if it was used recently.
- **TTL (Time To Live):** each key expires after a fixed duration regardless of usage. Best for time-sensitive data such as stock prices (TTL = 5 s) or OTP codes (TTL = 5 min).

5.3 Cache stampede (thundering herd)

PROBLEM / WATCH OUT

A viral key (e.g. a trending reel) expires. Suddenly **1 million simultaneous requests all miss** the cache and hit the database at once, which can overload or crash it.

Solution: mutex lock. When the key is missing, only **one** request acquires the lock, queries the DB and repopulates Redis. The others wait (or briefly retry), then read from the refreshed cache.

```
Without mutex: 1,000,000 requests -----> DB (overload!)
With mutex:    1,000,000 requests -> [lock] -> 1 request -> DB -> refill Redis
                                     999,999 wait -> read from Redis
```

5.4 Cold start and cache pre-warming

A brand-new cache is **empty**, so the first wave of users all miss and go to the database (this is the **cold start** problem). **Solution: pre-warm** the cache by loading popular data before taking live traffic.

EXAMPLE

Before a big sale starts, a script loads the top 1,000 products, the homepage banners and category lists into Redis. When customers arrive at 12:00 sharp, almost everything is already cached.

KEY TAKEAWAY

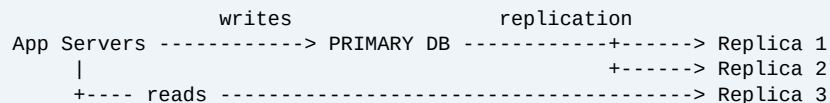
Redis = in-memory speed. Know: write-through vs write-back, LRU/LFU/TTL, stampede -> mutex, cold start -> pre-warm.

6. Database Replication

Now the product has 10 million users and one database cannot handle the traffic. Most apps have **far more reads than writes** (e.g. 10 million reads vs 1,000 writes).

6.1 Primary-replica architecture

- **Primary (leader/master) DB:** handles **all writes**.
- **Read replicas (followers):** receive copies of the data from the primary and handle **all reads**.
- The primary continuously replicates fresh data to every replica.



EXAMPLE

Instagram: millions of people scroll feeds (reads) while few post at any instant (writes). Posts go to the primary; feed reads are spread over replicas 1, 2, 3.

Why writes only go to the primary: writes must have a single source of truth. If two databases accepted writes independently they could conflict.

6.2 Failover and leader election

If the primary dies, the system performs **leader election**: one of the replicas is promoted to be the new primary and the remaining ones follow it. The service continues without long downtime.

6.3 Replication lag

Replication takes a little time (typically milliseconds). During that window the primary is **ahead** of replicas, so someone reading from a replica may see **stale data**.

EXAMPLE

You change your profile photo (written to primary). Within 5 ms you refresh and your read hits Replica 2, which has not received the update yet. You still see the old photo and think the update failed.

For other users, slightly stale data is usually fine. The writer, however, is confused. **Fix (read-your-own-writes):** after a user writes, route **that user's reads to the primary for a short window**. Once replicas catch up, switch back to replicas.

KEY TAKEAWAY

Replication scales **reads** and gives **fault tolerance**. It does NOT scale writes or total data size.

PROBLEM / WATCH OUT

New problem: data is now so large (100M+ rows) that even the primary and each replica (full copies!) cannot store or handle it. Answer: sharding.

7. Database Sharding and Consistent Hashing

Sharding = splitting data across multiple database servers, each holding a **subset**. Each shard is an independent database. In production we use **sharding + replication**: every shard has its own primary and read replicas.

```
Shard 1 (users A-M)           Shard 2 (users N-Z)
Primary 1                     Primary 2
|-- Replica 1a                |-- Replica 2a
+-- Replica 1b                +-- Replica 2b
(if Primary 1 dies, 1a or 1b is elected primary - same for shard 2)
```

7.1 How do you decide which record goes to which shard?

Range-based sharding

Split by ranges of the shard key (user ID, name, date).

EXAMPLE

Shard 1 stores users whose name starts with A-M; Shard 2 stores N-Z. Or user IDs 1-1,000,000 on Shard 1 and 1,000,001-2,000,000 on Shard 2.

- **Pro**: simple; range queries are efficient.
- **Con: hotspots**: if many names start with A-M or all new users go to the latest range, one shard is overloaded.

Hash-based sharding

Compute $\text{hash}(\text{key}) \bmod N$ where N is the number of shards.

EXAMPLE

With 5 shards, user ID 57: $57 \bmod 5 = 2$, so shard 2. User ID 51: $51 \bmod 5 = 1$, so shard 1. User ID 60: $60 \bmod 5 = 0$, so shard 0. IDs are spread evenly.

- **Pro**: even distribution.
- **Con**: if N changes (add/remove a shard), **almost all keys get remapped** and must be moved.

7.2 SQL vs NoSQL for sharding

- With **MySQL/PostgreSQL** you generally have to implement sharding yourself (routing logic in the app or a proxy).
- **NoSQL** databases such as **Cassandra** and **DynamoDB** are built to scale horizontally and handle sharding and replication automatically, but you give up some ACID guarantees (complex transactions/joins). Even then, interviewers expect you to understand how sharding works underneath.

7.3 The resharding problem and consistent hashing

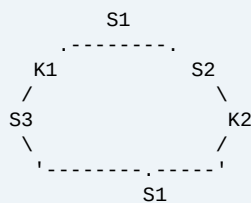
With plain hash $\bmod N$, changing N is very expensive.

EXAMPLE

Keys 1..12 on 3 servers (key mod 3): key 7 -> server 1. Add a 4th server (key mod 4): key 7 -> server 3. Roughly **75%** of keys change servers. Moving that much data causes huge load, and repeated server failures make it worse.

Consistent hashing fixes this:

- Both **servers and keys** are hashed onto a **virtual ring** (like a clock face).
- A key is stored on the **first server found going clockwise** from the key's position.
- When a server is added or removed, only the keys **between it and its predecessor** move, about **1/N** of all keys, not 75%.
- **Virtual nodes**: a single position per server gives uneven load. So each physical server is placed at **many** positions on the ring (S1, S2, S3, S1, S2, S3, ...). More virtual nodes means a more even distribution, and when a server dies its keys spread over many others instead of overloading one neighbor.



Key K1 -> walk clockwise -> first server = S2
If S2 dies, K1 simply goes to the next server clockwise (S3). Only S2's keys move.

Virtual nodes: S1, S2, S3 each appear many times around the ring for an even spread.

EXAMPLE

A cache cluster has 4 nodes. One node crashes. With mod-hashing nearly every key lands on a different node -> massive cache misses -> DB flooded. With consistent hashing only the crashed node's ~25% of keys are remapped; the other 75% of cache entries stay valid.

	hash mod N	Consistent hashing
Keys moved when adding a node	~ most (e.g. 75%)	~ 1/N
Distribution	Even (if N fixed)	Even with virtual nodes
Used by	Simple setups	Cassandra, DynamoDB, CDNs, distributed caches

KEY TAKEAWAY

Sharding scales **data size and writes**. Combine with replication for fault tolerance. Use consistent hashing so resharding is cheap.

8. Message Queues and Event-Driven Architecture

8.1 The problem: synchronous design

A user uploads a photo. The app server must resize it, compress it, generate a thumbnail, update the database and send a confirmation email. If all of this happens **synchronously** the user waits for everything to finish.

EXAMPLE

Resize 800 ms + compress 600 ms + thumbnail 400 ms + DB update 50 ms = **1,850 ms** just for those steps; add email and network time and the user stares at a spinner for 3-4 seconds, with no idea what is happening. If the email service is down, the whole upload can even fail.

8.2 The solution: decouple with a message queue

The app server **drops a job into a queue** and **immediately responds** to the user. Independent **workers** pick up jobs and process them in the background. This is **event-driven architecture**, and the message queue is its core.

```
User --upload--> App Server --(1) put job--> [ MESSAGE QUEUE ] --> Image Worker
                                     |
                                     |--> DB Worker
                                     |--> Email Worker
                                     +--> Analytics Worker
                                     +--(2) "Done!" in ~30 ms to user
```

8.3 Benefits (each with an example)

- **Fast response:** user gets 'upload complete' in ~30 ms instead of 3 seconds.
- **Decoupling and parallelism:** image, DB and email workers run **independently and at the same time**, not one-by-one.
- **Durability:** the job is stored in the queue, so it is not lost even if workers are temporarily down.
- **Traffic-spike buffering:** during a flash sale 100,000 orders arrive in a minute. The app keeps accepting them into the queue; workers drain it at their own pace. Users still get instant acknowledgment.
- **Retry / fault tolerance:** the Email worker crashes midway. The message returns to the queue and is retried automatically; the user never notices and the upload still shows as confirmed. If the Image worker was down, it comes back later, finds its pending job and continues.
- **Easy extensibility:** want logging/analytics? Just add a new Analytics worker that subscribes to the queue. No change to the app server is needed (in the synchronous design you would modify the upload code).

8.4 Kafka vs RabbitMQ

	Kafka	RabbitMQ
Model	Distributed, persistent event log	Traditional message broker
Retention	Messages kept for days/weeks/months	Typically removed after consumed
Consumers	Each tracks its own offset; can replay from any point	Messages delivered then acknowledged
Best for	Event streaming, real-time analytics, audit logs, huge throughput	Task queues, routing, job processing
Example	Track every click on Amazon and replay for a new analytics model	Send welcome emails, resize images

KEY TAKEAWAY

Queue = async processing, buffering, retries, decoupling. Kafka is more popular for streaming/replay use cases.

PROBLEM / WATCH OUT

New problem: we are storing photos and other big binary files in the database. That is a mistake. Answer: object storage.

9. Blob / Object Storage and Pre-signed URLs

9.1 Why not store files in the database?

Databases are optimized for **structured, queryable** data. Storing large binaries (images, videos, PDFs, audio) makes the DB **slow, expensive and hard to scale**.

Object storage such as **Amazon S3, Google Cloud Storage or Azure Blob Storage** is built precisely for this: cheap, durable and virtually unlimited. (S3 is just one popular example of blob storage.)

EXAMPLE

A 5 MB profile photo for 10 million users = 50 TB. Keeping that in MySQL would bloat backups and slow queries. In S3 it is cheap and the DB stores just a short string: `https://cdn.example.com/photos/u123.jpg`.

9.2 The pattern

- Client uploads a file (via the app server, or directly with a pre-signed URL).
- The file is stored in the **blob storage (S3 bucket)**.
- The file's **URL / key** is saved in the **database** and also placed in the **message queue** as a job.
- Workers read the job, fetch the file from S3, process it (resize, thumbnail, etc.).
- When users view the files, they are served through the **CDN**.

```
Client --> App Server --> S3 / Blob storage --URL--> Database (stores URL only)
|                                     \--> Message Queue --> Workers
+--> CDN --> Users (fast delivery)
```

9.3 Pre-signed URLs

For private files you should not expose the bucket publicly. Instead your server generates a **pre-signed URL**: a **temporary link with an embedded expiry time and cryptographic signature**. The client uses it to **upload or download directly to/from S3**, bypassing your server.

- **Saves server bandwidth and CPU**: big files do not pass through app servers.
- **Secure**: link expires (e.g. in 10 minutes) and the signature prevents tampering or reuse by others.
- Used by Dropbox, Google Drive and most file-sharing platforms.

EXAMPLE

Client asks: 'I want to upload report.pdf.' Server replies with a URL like `https://bucket.s3.amazonaws.com/report.pdf?Expires=...&Signature=...` valid for 10 minutes. The client PUTs the 200 MB file straight to S3. After 10 minutes the same link stops working.

KEY TAKEAWAY

Files -> object storage (S3). Database -> only metadata/URLs. Serve via CDN. Use pre-signed URLs for secure direct transfers.

10. Real-Time Communication: Polling, WebSockets, SSE

Standard HTTP is request-response: the client asks, the server answers, the connection closes. This is fine for normal pages, but not for **live chat, collaborative editing, live notifications, stock tickers**.

10.1 Short polling

The client repeatedly asks the server 'Any update?' at fixed intervals.

EXAMPLE

Every 2 seconds a chat app calls GET /messages. Server: 'No.' 'No.' 'No.' 'Yes, here is 1 message.'

Problem: most requests return nothing, so there is huge waste of resources and delayed updates.

10.2 Long polling

The client asks 'Any update?' and the server **holds the request open** until there is data (or a timeout), then responds; the client immediately asks again.

EXAMPLE

The client asks at 10:00:00; a message arrives at 10:00:07; the server answers at that moment and the client re-asks. Only one request per message instead of many empty ones.

Improvement over short polling, but still many open connections and repeated requests: not ideal at scale.

10.3 WebSockets

- A **single persistent, bidirectional** connection, established through a WebSocket handshake.
- Both sides can push messages at any time; very low overhead after the handshake.
- **Use cases:** chat apps, multiplayer games, collaborative editors (Google Docs), live trading.

EXAMPLE

In WhatsApp Web, your message is pushed to the server instantly over the open socket, and your friend's reply is pushed to you the moment it arrives. Neither side has to ask.

10.4 Server-Sent Events (SSE)

- A persistent connection that is **unidirectional**: only **server -> client**.
- Simpler than WebSockets and works over normal HTTP.
- **Use cases:** notification feeds, live sports scores, stock price tickers, progress bars.

EXAMPLE

A cricket score page opens an SSE connection. Each time a run is scored the server pushes the new score. The browser never sends data back on that channel.

Technique	Direction	Connection	Best for
Short polling	Client asks repeatedly	New each time	Simple, low-frequency checks
Long polling	Client asks, server holds	Re-opened per event	Fallback when sockets unavailable
WebSockets	Bidirectional	Persistent	Chat, games, collaboration
SSE	Server -> client only	Persistent	Notifications, feeds, tickers

11. Idempotency

An operation is **idempotent** if executing it **once or many times gives the same result**. GET, PUT and DELETE are naturally idempotent by design; POST is not, but can be made so with an **idempotency key**.

11.1 The problem

PROBLEM / WATCH OUT

A user pays \$100. The server charges the card, but the **network dies before the response reaches the client**. The client thinks it failed and **retries**. The server cannot tell it is a retry and charges again: the user is charged **\$200**.

The same issue affects order submission, email sending and database writes.

11.2 The solution: idempotency key

- The client generates a unique **UUID** before sending the request and puts it in a header, e.g. Idempotency-Key: pay_a3f9c2....
- The server checks (usually in Redis/DB) whether it has seen this key.
- **If found:** return the stored result without processing again.
- **If not found:** process the request and store the result under that key (often with an expiry).
- Used by Stripe, PayPal, Braintree and other payment APIs.

```
Attempt 1: POST /pay $100 key=A3F9C2 -> server charges, stores result -> response LOST
Attempt 2: POST /pay $100 key=A3F9C2 -> server finds key A3F9C2
                                         -> returns saved result, NO second charge
Final: user charged exactly $100, once.
```

KEY TAKEAWAY

Retries are inevitable in distributed systems; idempotency keys make them safe.

12. CAP Theorem

In a distributed system there are three desirable properties:

- **Consistency (C):** every read returns the most recent, correct data (never stale).
- **Availability (A):** every request receives a response, even if the data might be stale.
- **Partition tolerance (P):** the system keeps working even when the network between nodes breaks.

You **cannot have all three at once**.

12.1 Why CA is not a real choice

- Network partitions **will** happen in any distributed system, so you cannot ignore **P**. Ignoring it would mean 'if the network breaks, let my service break', which is unacceptable.
- So the real design choice is **CP or AP**.

12.2 CP vs AP

	CP (Consistency + Partition tolerance)	AP (Availability + Partition tolerance)
Philosophy	Correct data or no response	Always respond, even if stale
During a partition	May reject/err on some requests	Answers using whatever data the node has
Example	Banking, payments, inventory/stock counts, ticket booking	YouTube view counts, social feeds, likes, analytics
Outcome	Never shows wrong balance	Temporarily different values; eventually consistent

EXAMPLE

CP: your bank balance is Rs 10,000. During a network split, the bank refuses the withdrawal rather than risk letting two ATMs both dispense the same money.

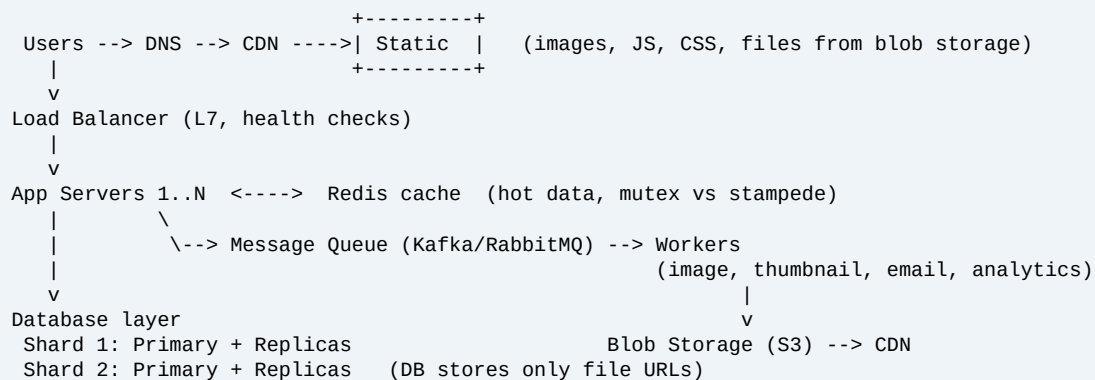
AP: your video really has 1,000,000 views. One user, reading from a lagging replica, sees 100K; another sees 200K. Both see the service working, and eventually all replicas converge to 1,000,000. Nobody is harmed by a slightly old count.

KEY TAKEAWAY

Decision rule: if stale data is unacceptable choose CP; if being always available matters more choose AP.

13. The Final Architecture

13.1 Complete picture



13.2 Request walkthrough: upload a photo and view it

- User opens the app: **DNS** resolves the domain; static assets (JS/CSS/images) load from the nearest **CDN** edge.
- API calls go over HTTPS to the **L7 load balancer**, which picks a healthy app server (e.g. least connections).
- User uploads a photo: app server (or a pre-signed URL) stores it in **S3**, saves the URL in the right **DB shard**, pushes a job to the **queue** and replies in ~30 ms.
- **Workers** independently resize, make thumbnails, send email and log analytics. Failed jobs are retried.
- Viewing the profile: the app checks **Redis** first; on a miss it reads from a **replica** of the correct shard and caches the result.
- The photo itself is delivered via the **CDN**, so it loads fast worldwide.
- If any server, replica or primary dies, health checks and **leader election** keep the system up.

13.3 Every component solves one problem

Problem	Component added	Key detail to mention
Single server can't cope / SPOF	Horizontal scaling	Vertical has hard limit + SPOF
Client can't choose a server	Load balancer (L7)	Round robin, least conn, IP hash; health checks
Far users have high latency	CDN	Edge nodes; TTL / versioned files

Problem	Component added	Key detail to mention
DB queries slow, DB overloaded	Redis cache	LRU/LFU/TTL; mutex; pre-warm
Too many DB reads / DB SPOF	Read replicas	Primary writes; leader election; lag fix
Data too large for one DB	Sharding (+replication)	Range / hash; consistent hashing
Slow work blocks user	Message queue + workers	Async, retries, spike buffer; Kafka
Files bloat the DB	Blob storage (S3)	Store URL only; pre-signed URLs; CDN
Live updates	WebSockets / SSE	Bidirectional vs server-push
Retries double-charge	Idempotency key	UUID in header; cached result
Stale vs correct data	CAP decision	CP for finance, AP for analytics

14. Interview Cheat Sheet

14.1 Quick Q&A

Q: Why not just keep making the server bigger?

A: Hardware has a ceiling and one server is a single point of failure. Scale horizontally.

Q: How does the client know which server to talk to?

A: It talks to a load balancer, which chooses a server using an algorithm and health checks.

Q: Round robin vs least connections?

A: Round robin for uniform requests; least connections when request durations vary.

Q: When use IP hashing?

A: When you need sticky sessions (same client to same server).

Q: L4 vs L7?

A: L4 routes on IP/port (fast, dumb). L7 understands URL, headers, cookies (smart).

Q: What is a CDN and what goes on it?

A: Globally distributed edge servers caching static content (images, JS, CSS, video), not dynamic API data.

Q: How do you invalidate CDN content?

A: TTLs, or versioned filenames like main.a1b2.js.

Q: Write-through vs write-back?

A: Write-through: safe but slower. Write-back: very fast but risks data loss if cache crashes.

Q: What is a cache stampede?

A: A hot key expires and a flood of requests hits the DB. Fix with a mutex so one request refills the cache.

Q: How do replicas help and what is the catch?

A: They scale reads and give failover; the catch is replication lag, fixed by reading from primary briefly after a write.

Q: Why consistent hashing?

A: Adding/removing nodes moves only $\sim 1/N$ keys instead of most keys; virtual nodes keep load even.

Q: Why use a queue?

A: Decouple and respond fast; workers process in background with retries and spike buffering.

Q: Where do you store images?

A: In object storage (S3), with only the URL in the DB; serve via CDN.

Q: WebSocket vs SSE?

A: WebSocket is bidirectional (chat). SSE is server-to-client only (notifications).

Q: How to prevent double payment?

A: Idempotency key (UUID) checked server-side.

Q: CAP for a bank vs YouTube views?

A: Bank: CP. YouTube views: AP (eventual consistency).

14.2 Notes on the source material

- The original transcript was auto-generated and contained a few garbled words (e.g. 'Radius' for Redis, 'item potency' for idempotency, 'short pulling' for polling); these notes use the correct terms.
- In the transcript's hash-sharding example the modulus results were stated loosely; here the arithmetic is shown correctly ($57 \bmod 5 = 2$, $51 \bmod 5 = 1$).
- The pre-signed URL flow is described in the standard way: the server issues the temporary URL and the client transfers the file directly to blob storage.